Bonsai Accelerator Implementation and Evaluation

Implementation report - Oriol Pont, July 2026

Scope

The benchmark report identified two Bonsai inference targets: Q1_0 by Q8_0 matrix-vector work and attention with K/V-cache traversal. The architecture blueprint then defined a shared NEORV32 Custom Functions Subsystem (CFS) shell, a compute engine for each target, and two data-delivery modes. `CPU_PUSH` is the straightforward interface where software relays payloads through CFS FIFOs. `MEM_STREAM` uses role-indexed descriptors and PSRAM bursts to deliver the same attention tiles with less CPU involvement.

This report evaluates the implemented RTL against the Tier 3 NEORV32 software baselines. Evaluation uses cycle-accurate NEORV32/CFS simulation and Gowin synthesis for the Tang Nano 9K target. The measured profiles preserve the Tier 3 operation shapes and checksums, and they establish operation-level acceleration.

Implemented System

Proposal	Profile	Implemented service
A	`Q1_MATVEC`	Packed 1-bit matrix weights (`-1/+1`) are dotted with signed 8-bit activation blocks. Each block result is rescaled using the weight and activation quantization scales, then all block results in a matrix row are accumulated, rounded, and returned as a signed 16-bit output.
B	`CPU_PUSH`	K/V append, QK scores, stable fixed-point softmax and weighted-V output, with CPU-serviced input and output FIFOs.
B	`MEM_STREAM`	The same attention engine with descriptor-driven transfers through a DQ16 Gowin PSRAM-controller boundary.

All profiles use the same command lifecycle and hardware-owned counter categories: command elapsed, engine elapsed, useful activity, input wait, output wait, control overhead, frontend waits, physical bytes and logical work. Proposal A compares every returned row value with its software reference. Proposal B compares a weighted checksum of the returned attention vector with the Tier 3 checksum and also validates K/V append data, traffic, work, and cycle identities before accepting a run.

Evaluation method

The firmware prepares each fixture before timing, launches one CFS service and validates its output. Command gain divides Tier 3 software-service cycles by complete hardware command cycles. Active-cycle gain isolates useful engine work from data-delivery overhead. Both firmwares use the same RV32I target, operation fixtures and NEORV32 simulator.

`MEM_STREAM` models the generated Gowin PSRAM HS controller interface: DQ16 physical width, 64-bit user beats, 32-byte bursts, six cycles of configured read latency and an 18-user-clock minimum command interval. Fixtures are preloaded before timing, and synthesis assumes command inputs already reside in PSRAM. The model covers controller timing and handshakes; wider-system initialization and electrical pin behavior remain outside the measured service.

Board feasibility uses Gowin FPGA Designer Education V1.9.11.03 for `GW1NR-LV9QN88PC6/I5` at 27 MHz. Each trimmed SoC omits the other engine and unused frontend. Proposal A and Proposal B `CPU_PUSH` use the applicable board pin constraints; over-capacity `MEM_STREAM` stops before pin placement and timing analysis.

The cycle results are preserved under `results/proposal_a_evaluation/` and `results/proposal_b_evaluation/`. Raw Gowin resource, timing and capacity reports are preserved under `results/gowin_synthesis/`.

Proposal A: Q1/Q8 Matvec

Profile	Tier 3	Command	Active	Cmd. gain	Util.
Board, `1 x 128`	7,934	1,804	82	4.398x	4.843%
Bonsai, `1 x 2048`	195,602	27,863	1,297	7.020x	4.673%

Both outputs match the Tier 3 checksums. The packed arithmetic reduces useful work to 82 cycles for one 128-element group and 1,297 cycles for sixteen groups. Complete command speedup is lower because `CPU_PUSH` sends every packed activation and weight tile through CFS. Even with that straightforward path, the service remains 4.4x to 7.0x faster than the naive software implementation.

Proposal B: Attention/KV

Profile	Tier 3	`CPU_PUSH`	`MEM_STREAM`	CPU gain	Stream/CPU
Board, `H1/KVH1/D32/C2`	489,007	5,453	810	89.677x	6.732x
GQA, `H2/KVH1/D16/C2`	494,741	4,874	706	101.506x	6.904x

Profile	Active	CPU util.	Stream util.	Frontend wait CPU → stream	Checksum
Board	398	8.707%	51.756%	4,012 → 169	5,274
GQA	412	9.383%	59.624%	3,284 → 118	7,569

The CPU-push implementation is already 89.677x and 101.506x faster than the corresponding Tier 3 services. Its engine utilization remains below 10%, showing that FIFO delivery dominates the remaining command time. With the same 398 and 412 active engine cycles, `MEM_STREAM` reduces command time by another 6.7x to 6.9x, raises utilization above 51%, and sharply reduces frontend input wait. Both transfer modes produce the displayed expected output checksums.

Note that these profiles validate service compatibility and memory-path accounting at just `ctx = 2`, given the physical PSRAM controller and local score storage restrictions.

Tang Nano 9K Feasibility

Profile	Logic	Registers	BSRAM	DSP	Fmax
Proposal A	6,758 / 8,640	3,329 / 6,693	16 / 26	8 / 10	32.678 MHz
Proposal B `CPU_PUSH`	7,890 / 8,640	4,331 / 6,693	10 / 26	3.5 / 10	28.377 MHz
Proposal B `MEM_STREAM`	11,578 / 8,640	-	-	-	-

Proposal A and Proposal B CPU push both complete place and route above the required 27 MHz clock, with zero setup and hold total negative slack. They use 79% and 92% of the available logic, respectively. Proposal B CPU push also occupies 98% of the available configurable logic slices, so its successful placement has limited resource margin. The MEM_STREAM-only profile excludes the CPU FIFO and software memory aperture, yet the NEORV32 SoC, attention engine, descriptor streamer, PLL and generated DQ16 PSRAM controller require 134% of device logic.

Conclusions

The project demonstrates two hardware services derived from measured Bonsai workloads and validated on the selected operation fixtures. Proposal A turns packed Q1/Q8 computation into an overall 4.4x to 7.0x service speedup and fits the target FPGA. Proposal B moves the implemented attention operation into hardware; its board-feasible CPU-push version is about 90x to 102x faster than naive software. Descriptor-driven streaming then removes most frontend waiting in controller-contract simulation and provides a further 6.7x to 6.9x improvement around the same compute engine.

However, the MEM_STREAM implementation exceeds the available logic and cannot be placed on the Tang Nano 9K. Not only that, but even these reduced accelerator services can't both fit simultaneously on the same device. This means that we can't make confident assertions and extrapolations about the performance of a complete Bonsai LLM accelerator on the Tang Nano 9K, which has proved impossible. If using a larger FPGA, the MEM_STREAM service could be implemented and evaluated, both proposals could be placed on the same device and tuned, and the complete Bonsai LLM accelerator could be implemented and evaluated, to end up assessing real token throughput gain with respect to the original LLM CPU baseline.